```
import pandas as pd
import numpy as np
df = pd.read_csv('ReadMe.csv', on_bad_lines='skip', encoding='utf-8')
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipykernel_2331/3225641313.py in <cell line: 0>()
      1 import pandas as pd
      2 import numpy as np
----> 3 df = pd.read_csv('ReadMe.csv', on_bad_lines='skip', encoding='utf-8')

                                 4 frames
/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in get_handle(path_or_buf,
mode, encoding, compression, memory_map, is_text, errors, storage_options)
    880         else:
    881             # Binary mode
--> 882             handle = open(handle, ioargs.mode)
    883         handles.append(handle)
    884

FileNotFoundError: [Errno 2] No such file or directory: 'ReadMe.csv'
```

```
# 1. Bring pandas into Python's memory so it is defined
import os
import pandas as pd
import numpy as np

# 2. Check if the file is already there, if not, open a file picker
if not os.path.exists('ReadMe.csv'):
    print("🔄 'ReadMe.csv' not found in memory. Opening mobile file selector...")
    try:
        from google.colab import files
        uploaded = files.upload()  # This opens your phone's file browser
    except Exception as e:
        print("If you are not on Google Colab, please manually upload 'ReadMe.csv' usin
else:
    print("✅ 'ReadMe.csv' is already uploaded and ready!")

# 3. Read the data safely now that pd and the file are both ready
try:
    df = pd.read_csv('ReadMe.csv')
```

```
    print("🎉 Success! The dataset loaded perfectly. Here is a preview:")
    print(df.head(2))
except Exception as e:
    print(f"❌ Could not read the file yet. Error details: {e}")
```

```
🔄 'ReadMe.csv' not found in memory. Opening mobile file selector...
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving ReadMe.csv to ReadMe.csv
🎉 Success! The dataset loaded perfectly. Here is a preview:
     fg    3p    ft  reb  ast  stl  blk  tov  target_5yrs  total_points  \
0  34.7  25.0  69.9  4.1  1.9  0.4  0.4  1.3            0         266.4
1  29.6  23.5  76.5  2.4  3.7  1.1  0.5  1.6            0         252.0

   efficiency
0    0.270073
1    0.267658
```

```
# ======================================================================
# TASK 1: DATA EXPLORATION, TARGET PROFILE & CLASS BALANCE CHECK
# ======================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, precision_score, recall_score, classifica

# Load the engineered dataset
df = pd.read_csv('ReadMe.csv')
print(f"Dataset loaded successfully. Dimensions: {df.shape}")

# Confirm and print class baseline distributions
target_counts = df['target_5yrs'].value_counts()
target_pct = df['target_5yrs'].value_counts(normalize=True) * 100

print("\n--- Target Variable 'target_5yrs' Profile ---")
for cls in target_counts.index:
    status = "5+ Years (Longevity)" if cls == 1 else "< 5 Years (Bust)"
    print(f" Class {cls} [{status}]: {target_counts[cls]} rows ({target_pct[cls]:.2f}%)
```

```
Dataset loaded successfully. Dimensions: (1340, 11)

--- Target Variable 'target_5yrs' Profile ---
 Class 1 [5+ Years (Longevity)]: 831 rows (62.01%)
 Class 0 [< 5 Years (Bust)]: 509 rows (37.99%)
```

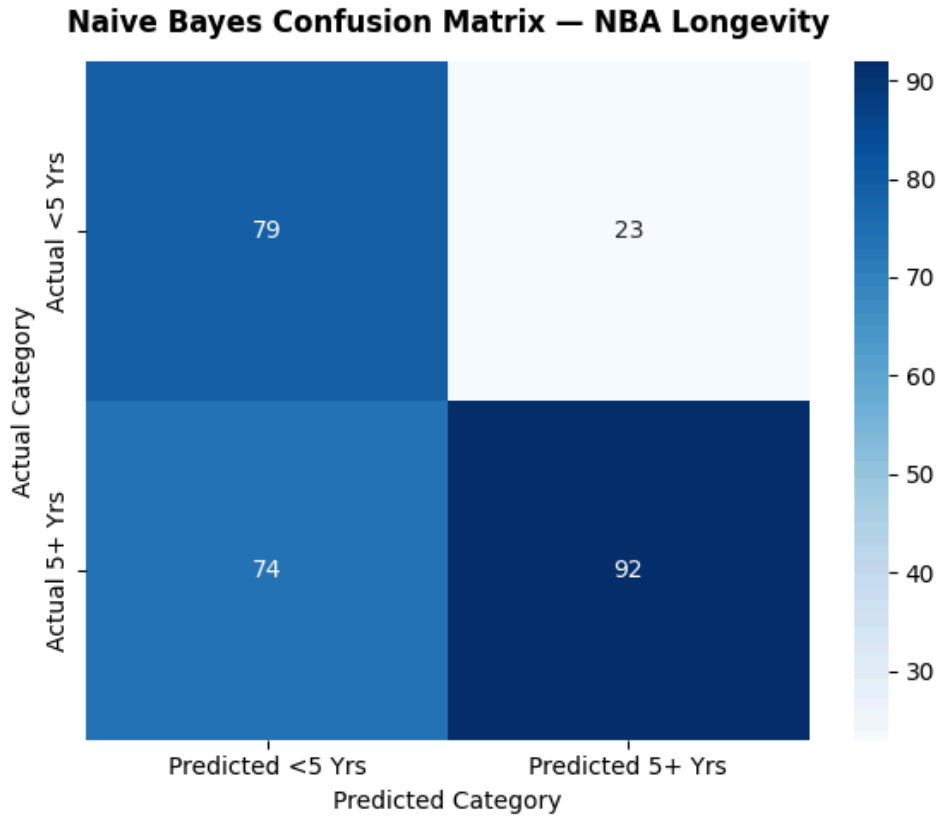
```
# ======================================================================
# TASK 2: TRAIN-TEST SPLIT & GAUSSIAN NAIVE BAYES IMPLEMENTATION
# ======================================================================

# Separate features from target (all columns here are continuous stats)
X = df.drop(columns=['target_5yrs'])
y = df['target_5yrs']

# 80/20 train-test split with stratification to maintain class proportions
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training matrices shape : {X_train.shape}")
print(f"Testing matrices shape  : {X_test.shape}")

# Initialize and fit the Gaussian Naive Bayes Classifier
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
print("\n✅ Gaussian Naive Bayes model trained successfully.")
```

```
Training matrices shape : (1072, 10)
Testing matrices shape  : (268, 10)

✅ Gaussian Naive Bayes model trained successfully.
```

```
# ======================================================================
# TASK 3: METRICS EXTRACTION & MATRIX VISUALIZATION
# ======================================================================

# Generate binary predictions on the test set
y_pred = nb_model.predict(X_test)

# Calculate precision and recall scores
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
raw_cm = confusion_matrix(y_test, y_pred)

print("--- Core Performance Metrics ---")
print(f"Precision Score (Minimizes Draft Busts)  : {precision:.4f}")
print(f"Recall Score (Minimizes Missed Talent) : {recall:.4f}\n")
print("Detailed Classification Report:\n", classification_report(y_test, y_pred))

# Render Confusion Matrix Heatmap using Seaborn
plt.figure(figsize=(6, 5))
sns.heatmap(
    raw_cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Predicted <5 Yrs', 'Predicted 5+ Yrs'],
    yticklabels=['Actual <5 Yrs', 'Actual 5+ Yrs']
)
plt.title('Naive Bayes Confusion Matrix — NBA Longevity', fontsize=12, fontweight='bold'
plt.ylabel('Actual Category')
plt.xlabel('Predicted Category')
plt.tight_layout()
plt.savefig('nb_confusion_matrix.png', dpi=150)
plt.show()
```

```
--- Core Performance Metrics ---
Precision Score (Minimizes Draft Busts)  : 0.8000
Recall Score (Minimizes Missed Talent) : 0.5542

Detailed Classification Report:
              precision    recall  f1-score   support

           0       0.52      0.77      0.62       102
           1       0.80      0.55      0.65       166

    accuracy                           0.64       268
   macro avg       0.66      0.66      0.64       268
weighted avg       0.69      0.64      0.64       268
```




```
# ======================================================================
# TASK 4: INSIGHT EXTRACTION — FEATURE DISTRIBUTION PER CLASS
# ======================================================================

# Extract feature means conditioned on class labels (theta_ attribute)
feature_means = pd.DataFrame(
    nb_model.theta_,
    columns=X.columns,
    index=['Class 0 (<5 Yrs)', 'Class 1 (5+ Yrs)']
).T

print("--- Gaussian Means Matrix (Feature Tracking) ---")
print(feature_means.round(4))
```

```
--- Gaussian Means Matrix (Feature Tracking) ---
     Class 0 (<5 Yrs)  Class 1 (5+ Yrs)
fg            42.4280          45.2024
3p            18.5877          18.9794
ft            68.8398          71.0823
reb            2.2138           3.5359
```

```
ast                    1.2233          1.7874
stl                    0.4916          0.6997
blk                    0.2489          0.4408
tov                    0.9356          1.3672
total_points         282.9602        560.8063
efficiency             0.3493          0.3871
```

```
Start coding or generate with AI.
```